

Wazuh + YARA Integration

Automated Malware Detection & Auto-Removal

Complete Step-by-Step Setup Guide

Author: Rakibul Islam Joy

Security Engineer | Unified IT

github.com/rakibuljoy

June 2026

Overview

This guide covers extending an existing Wazuh + YARA malware detection setup with a fully automated removal stage. Written from a real working environment — every command in this guide was tested and confirmed working, and the debugging story (Pages 3–4) is included exactly as it happened, so the reasoning behind each decision is clear.

What this setup does:

- FIM (File Integrity Monitoring) watches directories for new or changed files in real time
- A file change automatically triggers Active Response #1, which runs a YARA scan on that exact file
- If YARA matches a known malware signature, Wazuh fires a Level 15 alert (rule 108001)
- Active Response #2 runs **remove-malware.sh** on the agent — the infected file gets deleted automatically
- Every action (deleted, not found, or extraction error) is logged for a full audit trail

Environment used:

- Wazuh Manager: Ubuntu Server (hostname: **wazuh-soc**)
- Wazuh Agent: Ubuntu (hostname: **rakib-virtual-machine**, Wazuh agent name: Ubuntu-Server, agent ID 002)
- YARA installed on the agent with malware signature rules (tested using the standard EICAR test signature)

NOTE: This guide assumes YARA is already installed on the agent and that Wazuh already scans files on FIM events (Stage 1, Page 5), with a custom rule (108001) already firing on a YARA match. If that part isn't in place yet, set it up first — this guide's main focus is **Stage 2: fully automated removal**.

The Real Debugging Journey — Part 1

This isn't a cleaned-up summary — it's what actually happened while building this, including the dead end before the fix. Kept here so the next implementation (by anyone) skips straight past the same trap.

DESIGN DECISION

Active Response needs three pieces working together

A `<command>` definition on the manager (names the script), an `<active-response>` binding (decides which rule triggers it), and the actual script sitting on the agent (which runs locally and does the deletion). The binding was deliberately pointed at rule **108001** — the alert that fires only *after* YARA has already confirmed a match — so removal always happens strictly after detection, never before.

ATTEMPT 1 — FAILED

First script version used jq to read the alert JSON

This looked correct on paper — straightforward JSON field access:

```
MALWARE_FILE=$(echo "$INPUT_JSON" | jq -r '.parameters.alert.data.yara.file')
```

Config was added, services restarted, EICAR test file created — but two problems showed up.

BLOCKER 1

chmod / chown: "Permission denied"

Running the permission commands directly as a normal user failed, because everything under `/var/ossec/` is owned by **root:wazuh** — including the script file itself, which also couldn't be created without `sudo` in the first place.

FIX

Re-ran both the file creation (`sudo nano ...`) and the permission commands with `sudo`.

BLOCKER 2

Script ran, but nothing got deleted

After fixing permissions and restarting both services, the EICAR test correctly triggered the active response — confirmed in **active-responses.log** — but every entry read:

```
File not found (already removed?):
```

No file path at all. The chain was firing correctly end to end; the script just couldn't read the path out of the JSON. Continued on the next page.

The Real Debugging Journey — Part 2

DIAGNOSING IT

Added a temporary debug line to see the raw alert

Instead of guessing, the actual JSON Wazuh sends via stdin was captured directly — right after reading the input:

```
read INPUT_JSON
echo "$INPUT_JSON" >> /tmp/ar_debug.log
```

A fresh EICAR file was created to trigger the chain again, then the captured file was inspected: **sudo cat /tmp/ar_debug.log**

ROOT CAUSE FOUND

Wazuh's own alert JSON is technically malformed

The captured payload showed the cause: Wazuh embeds the original scanner log line inside a **full_log** field as a string — and that inner string has its own un-escaped double quotes. That makes the overall outer JSON invalid. **jq** doesn't throw a visible error on this; it just silently returns an empty string, which is exactly why **MALWARE_FILE** kept coming back blank with no obvious clue why.

THE FIX

Replaced jq with a direct sed pattern match

Targets the exact **"data":{"yara":{"...","file":"..."}}** segment directly, without needing the rest of the JSON to be valid:

```
MALWARE_FILE=$(echo "$INPUT_JSON" | sed -n 's/.*"data":{"yara":{"scan":"[^"]*", "matched_rule":"[^"]*", "file":"\"([^\"]*)\"\".*$/\1/p')
```

Re-tested with a fresh EICAR file. This time the log showed **"Deleted malicious file: ..."**, and **ls** confirmed the file was actually gone.

CLEANUP

Removed the debug line once the fix was confirmed

The debug line (**echo "\$INPUT_JSON" >> /tmp/ar_debug.log**) was only ever meant for diagnosis. Once the sed-based extraction was confirmed working, it was deleted from the script and **/tmp/ar_debug.log** itself was removed — leaving it in would just keep writing full alert payloads to disk indefinitely, serving no purpose once the bug was found. **The script in Step 3 (Page 7) is this final, debug-free version.**

One more thing noticed during testing: the same file sometimes logged both "Deleted malicious file" and "File not found (already removed?)" back to back. That's not a bug — two separate YARA signatures ("EICAR_Test_File" and "eicar") both matched the same test file, so rule 108001 fired twice for one event, and the active response simply ran twice. The script's existence check already handles this safely — see Troubleshooting, Page 11.

Step 1 — Confirm Stage 1 Is In Place (FIM → YARA Scan)

This stage should already be configured from your existing YARA setup. It's listed here for reference, so the full chain is documented end-to-end before Step 2 adds the new removal stage.

On the **MANAGER**, in `/var/ossec/etc/ossec.conf`, you should already have:

```
# Run this on: MANAGER (wazuh-soc)

<command>
  <name>yara_scan</name>
  <executable>yara_scan.sh</executable>
  <timeout_allowed>no</timeout_allowed>
</command>

<active-response>
  <command>yara_scan</command>
  <location>local</location>
  <rules_id>554,553,552,550</rules_id>  <!-- FIM rule IDs -->
  <timeout>60</timeout>
</active-response>
```

NOTE: Rules 550–554 are Wazuh's built-in FIM alert IDs. This means: whenever FIM detects a file add or change, Wazuh automatically runs a YARA scan on that exact file via **yara_scan.sh** — before any malware verdict exists yet. The result is written to `/var/ossec/logs/yara_scan.log`, which a custom rule (108001) parses to raise the actual malware alert.

Step 2 — Add the Auto-Removal Command + Active Response

This is the new stage this guide adds. It binds a second Active Response directly to rule **108001** — the alert that fires only after YARA has already confirmed a malware match.

```
# Run this on: MANAGER (wazuh-soc)
sudo nano /var/ossec/etc/ossec.conf
```

Add this block before **</ossec_config>** — below your existing `yara_scan` blocks, not replacing them:

```
<!-- 1. Command definition -->
<command>
  <name>yara_remove</name>
  <executable>remove-malware.sh</executable>
  <timeout_allowed>no</timeout_allowed>
</command>

<!-- 2. Active response – triggers only when rule 108001 fires -->
<active-response>
  <command>yara_remove</command>
  <location>local</location>
  <rules_id>108001</rules_id>
</active-response>
```

WARNING: Keep both active-response blocks in the same file. Wazuh allows multiple `<command>` + `<active-response>` pairs as long as each `<command>` has a unique `<name>`. Deleting the Stage 1 block to "make room" for this one will break detection entirely.

Step 3 — Create the Removal Script on the AGENT

The manager only sends the command — the script must physically exist on the agent machine, which runs it locally to delete the file. This is the **final version** — sed-based, with the diagnostic debug line already removed (see the Journey, Pages 3–4, for why).

```
# Run this on: AGENT (rakib-virtual-machine)
sudo nano /var/ossec/active-response/bin/remove-malware.sh
```

Paste the complete script below:

```
#!/bin/bash
# Wazuh Active Response – YARA Malware Removal Script
# Deletes a file once YARA (via rule 108001) confirms it is malware.

LOG_FILE="/var/ossec/logs/active-responses.log"
read INPUT_JSON

# Extract the file path with sed, not jq – see Journey, Page 4
MALWARE_FILE=$(echo "$INPUT_JSON" | sed -n 's/.*"data":{"yara":{"scan":"[^"]*", "matched_rule":"[^"]*", "file":"\"([^\"]*)\".*\\1/p'})

# Decode URL-encoded characters (e.g. %3A, %2F)
MALWARE_FILE=$(python3 -c "import urllib.parse,sys; print(urllib.parse.unquote(sys.argv[1]))"
"$MALWARE_FILE")

if [ -n "$MALWARE_FILE" ] && [ -f "$MALWARE_FILE" ]; then
    rm -f "$MALWARE_FILE"
    echo "$(date) - Deleted malicious file: $MALWARE_FILE" >> "$LOG_FILE"
elif [ -z "$MALWARE_FILE" ]; then
    echo "$(date) - ERROR: Could not extract file path from JSON" >> "$LOG_FILE"
else
    echo "$(date) - File not found (already removed?): $MALWARE_FILE" >> "$LOG_FILE"
fi

exit 0
```

CRITICAL — why sed, not jq: Wazuh's alert JSON embeds the raw scanner log line inside a **full_log** field as a string, and that inner string has its own unescaped quotes. This makes the overall JSON technically invalid. Strict parsers like **jq** don't error here — they just silently return nothing, which is the hardest kind of failure to debug. Pattern-matching the exact field with **sed** avoids this problem entirely. Full story on Page 4.

Step 3 (continued) — Set Permissions

Set correct ownership and permissions on the **AGENT**:

```
# Run this on: AGENT (rakib-virtual-machine)
sudo chmod 750 /var/ossec/active-response/bin/remove-malware.sh
sudo chown root:wazuh /var/ossec/active-response/bin/remove-malware.sh

# Verify
sudo ls -la /var/ossec/active-response/bin/remove-malware.sh
# Expected: -rwxr-x-- 1 root wazuh ... remove-malware.sh
```

WARNING: "Permission denied" on chmod/chown almost always means the **sudo** was missing — this is exactly what happened the first time (Page 3). Everything under **/var/ossec/** is owned by **root:wazuh** and is not writable by a normal user account, including creating the script file itself.

Step 4 — Validate Config and Restart

Always validate before restarting — this catches XML errors without crashing the manager.

```
# Run this on: MANAGER (wazuh-soc)
sudo /var/ossec/bin/wazuh-control configtest

# If clean, restart both manager and agent
sudo systemctl restart wazuh-manager
sudo systemctl status wazuh-manager
# Look for: Active: active (running)

# Run this on: AGENT (rakib-virtual-machine)
sudo systemctl restart wazuh-agent
sudo systemctl status wazuh-agent
```


Step 5 — Test the Full Flow with EICAR

EICAR is a harmless, industry-standard test string that any malware scanner (including YARA's default test rule) recognizes as malicious. It will not harm your system — it exists specifically to test detection pipelines like this one.

On the **AGENT** machine, create the test file inside a monitored directory:

```
# Run this on: AGENT (rakib-virtual-machine)
echo 'X50!P%@AP[4\PZX54(P^)7CC)7}$EICAR-STANDARD-ANTIVIRUS-TEST-FILE!$H+H*' >
/home/rakib/eicar-test.txt

ls -la /home/rakib/eicar-test.txt
```

Watch the active response log (wait a few seconds):

```
# Run this on: AGENT
sudo tail -f /var/ossec/logs/active-responses.log
```

Successful output will look like this:

```
Tue Jun 16 12:30:49 PM +06 2026 - Deleted malicious file: /home/rakib/eicar-test.txt
```

Confirm the file is gone:

```
ls /home/rakib/eicar-test.txt
# Expected: ls: cannot access '/home/rakib/eicar-test.txt': No such file or directory
```

Check the Wazuh Dashboard: go to **Security Events** and filter by **rule.id: 108001** — you should see the matching alert at Level 15.

Troubleshooting

chmod / chown: "Permission denied"

Cause: command was run as a normal user, without `sudo`.

Fix: re-run both the file creation and the permission commands with **sudo**.

Active response fires, but "File not found (already removed?)" shows no path at all

Cause: the script was using `jq` to parse the alert JSON. Wazuh's **full_log** field contains unescaped inner quotes, making the JSON technically invalid — `jq` silently returns an empty string instead of erroring.

Fix: use the **sed**-based extraction shown in Step 3, which doesn't require valid JSON.

Same file shows both "Deleted" and "File not found (already removed?)" back-to-back

Cause: this is expected, not an error. It happens when more than one YARA rule matches the same file (e.g. both "EICAR_Test_File" and "eicar" signatures match), so rule 108001 — and the active response — fires twice for one event.

Fix: no fix needed. The script's existence check (`[ -f "$MALWARE_FILE" ]`) already handles this gracefully.

wazuh-manager fails to start after editing ossec.conf

Always validate first: **sudo /var/ossec/bin/wazuh-control configtest** — read the error message and fix the line it points to before restarting.

Active response log shows nothing at all after the test

Cause: **remove-malware.sh** is missing on the agent, or the `rule_id` binding doesn't match 108001.

Fix: confirm the script exists on the agent (Step 3), then check: **sudo grep "yara_remove" /var/ossec/logs/ossec.log**

Quick Reference

Key File Paths

File / Component	Path
Manager main config	/var/ossec/etc/ossec.conf
Removal script (on AGENT)	/var/ossec/active-response/bin/remove-malware.sh
YARA scan log	/var/ossec/logs/yara_scan.log
Active response log (on AGENT)	/var/ossec/logs/active-responses.log
Manager error log	/var/ossec/logs/ossec.log

Useful Commands

What it does	Command
Open main config	sudo nano /var/ossec/etc/ossec.conf
Validate config (no restart)	sudo /var/ossec/bin/wazuh-control configtest
Restart manager	sudo systemctl restart wazuh-manager
Restart agent	sudo systemctl restart wazuh-agent
Watch active response log	sudo tail -f /var/ossec/logs/active-responses.log
Check manager errors	sudo grep "yara_remove" /var/ossec/logs/ossec.log

Final Setup Checklist

Go through this every time you set this up on a new agent:

Task	Done?
Stage 1 (yara_scan) command + active-response already configured	☐
Stage 2 (yara_remove) command + active-response added — without removing Stage 1	☐
remove-malware.sh created on the AGENT (not just the manager)	☐
remove-malware.sh uses sed-based extraction, not jq	☐
Debug line (echo \$INPUT_JSON >> /tmp/ar_debug.log) NOT present in final script	☐
remove-malware.sh permissions: 750, owner root:wazuh (on agent)	☐
sudo wazuh-control configtest returns no errors	☐
wazuh-manager and wazuh-agent both show Active: active (running)	☐
EICAR test file was auto-deleted by active response	☐
Rule 108001 alert visible in Wazuh Dashboard	☐

-- End of Document --